

# Optimisation Numérique et Sciences des Données

Sciences des données – Introduction aux réseaux de neurones

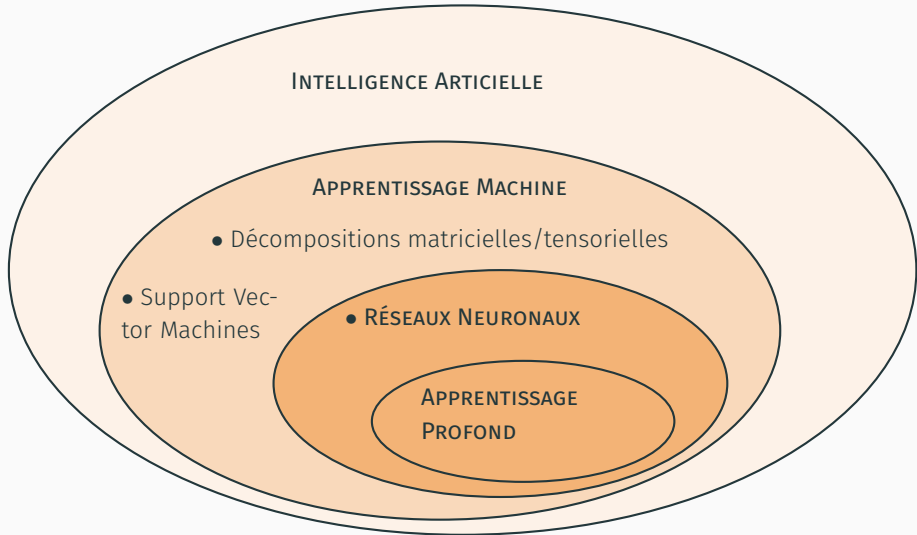
---

Paul Catala (CRAN, UL)  
paul.catala(at)univ-lorraine.fr

Master Ingénierie des Systèmes Complexes M2 ISC 925  
Semestre 9 – 2024

- 
- Connaître quelques outils fondamentaux (différentiation automatique)
- Comprendre les difficultés numériques (coût calculatoire, vanishing gradient)
- Observer quelques architectures importantes

1. Introduction
2. Modèle linéaire
3. Réseaux multi-couches
4. Différentiation automatique

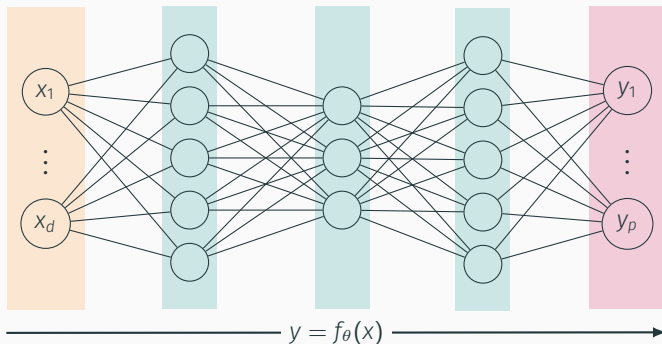


- **Réseau de neurones** : ensemble de couches de "neurones" **apprenant** comment transmettre l'information d'une couche à l'autre à partir des données, afin de réaliser une tâche spécifique (classification, prédiction, traduction, génération, etc...)

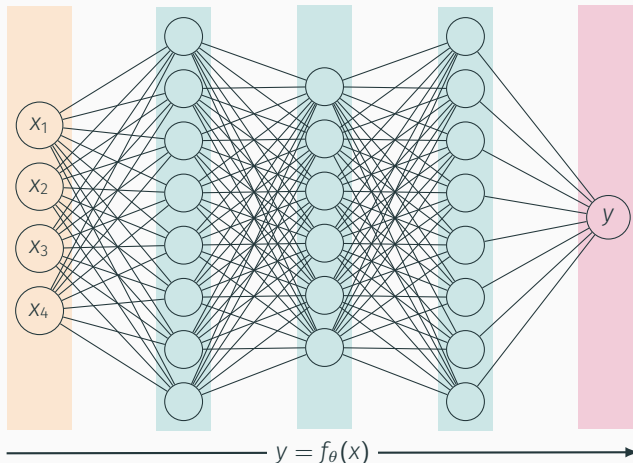
Couche d'entrée

Couches latentes (ou cachées)

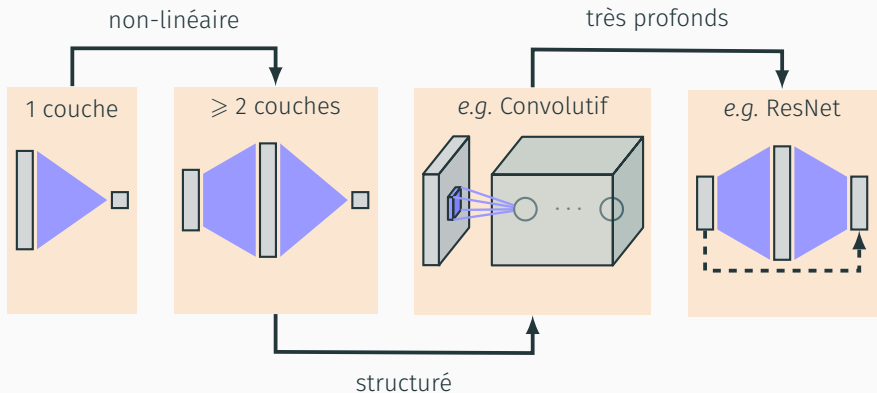
Couche de sortie



- Différentes tâches = différentes architectures de réseau, différents critères



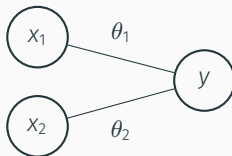
**Classification:** minimiser l'erreur  $\sum_j \ell(f_{\theta}(x^{(j)}), y^{(j)})$  sur des données  $(x^{(j)}, y^{(j)})$   
Variables:  $\sim 10^6$  à  $10^9$  paramètres  $\theta$



1. Introduction
2. Modèle linéaire
3. Réseaux multi-couches
4. Différentiation automatique



- Un réseau de neurones est un **graphe de calcul**: les connections représentent des opérations mathématiques de base

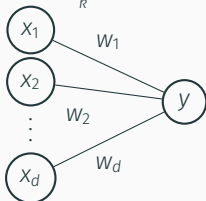


- Sur cette représentation simple:
  - à chaque connection est affecté un poids **multiplicatif**:  $\theta_1 x_1$ ,  $\theta_2 x_2$
  - un neurone de sortie **additionne** les entrées
- On a donc représenté la fonction

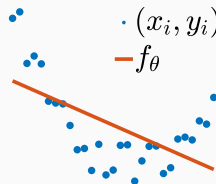
$$y = f_{\theta}(x) = \theta_1 x_1 + \theta_2 x_2$$

- Toute **fonction linéaire** peut se modéliser comme un réseau à une couche

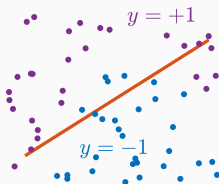
$$f_{\theta}(x) = \theta^{\top} x = \sum_k x_k w_k$$



Regression:  $y = f_{\theta}(x)$



Classification:  $f_{\theta}(x) = 0$



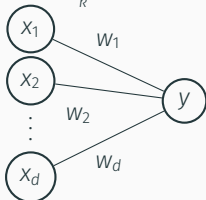
- Critère d'optimisation: à partir de  **$n$  données labellisées**  $(x^{(i)}, y^{(i)}) \in \mathbb{R}^d \times \mathcal{Y}$

$$\min_{\theta \in \mathbb{R}^d} \sum_{i=1}^n \ell(\theta^{\top} x^{(i)}, y^{(i)})$$

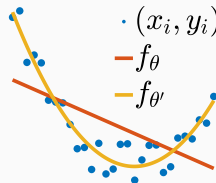
où  $\ell$  est une fonction perte adéquate.

- Toute **fonction linéaire** peut se modéliser comme un réseau à une couche

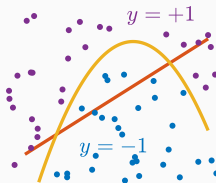
$$f_{\theta}(x) = \theta^{\top} x = \sum_k x_k w_k$$



Regression:  $y = f_{\theta}(x)$



Classification:  $f_{\theta}(x) = 0$



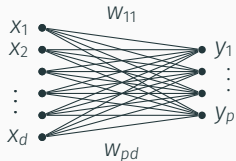
- Critère d'optimisation: à partir de  **$n$  données labellisées**  $(x^{(i)}, y^{(i)}) \in \mathbb{R}^d \times \mathcal{Y}$

$$\min_{\theta \in \mathbb{R}^d} \sum_{i=1}^n \ell(\theta^{\top} x^{(i)}, y^{(i)})$$

où  $\ell$  est une fonction perte adéquate.

- **Remarque:**  $\theta \mapsto f_{\theta}(x)$  est linéaire, mais pas nécessairement  $x \mapsto f_{\theta}(x)$ :
  - **Méthodes à noyau:** remplacer  $x$  par  $\varphi(x) \in \mathbb{R}^D$ , avec  $D \gg d$
  - **Apprentissage profond:** introduire de la non-linéarité pour apprendre  $\varphi$

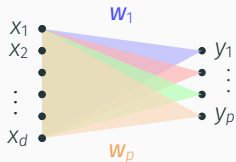
- Si la sortie est vectorielle



$$y_j = \sum_k w_{jk} x_k = \mathbf{w}_j^\top \mathbf{x}$$

$$\mathbf{y} = \mathbf{W}\mathbf{x}, \quad \mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots \\ w_{21} & w_{22} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

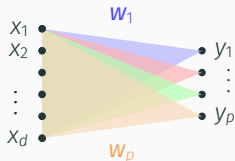
- Si la sortie est vectorielle



$$y_j = \sum_k w_{jk} X_k = \mathbf{w}_j^T \mathbf{X}$$

$$\mathbf{y} = \mathbf{W}\mathbf{x}, \quad \mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots \\ w_{21} & w_{22} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^T \\ \mathbf{w}_2^T \\ \vdots \\ \mathbf{w}_p^T \end{bmatrix}$$

- Si la sortie est vectorielle



$$y_j = \sum_k w_{jk} x_k = \mathbf{w}_j^T \mathbf{x}$$

$$\mathbf{y} = \mathbf{W}\mathbf{x}, \quad \mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots \\ w_{21} & w_{22} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^T \\ \mathbf{w}_2^T \\ \vdots \\ \mathbf{w}_p^T \end{bmatrix}$$

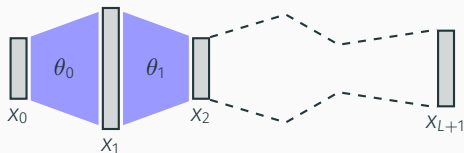
- Plus généralement, on peut modéliser une fonction affine, *i.e.*

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}.$$

$\theta := (\mathbf{W}, \mathbf{b})$  sont les paramètres du réseau (pour une couche).

1. Introduction
2. Modèle linéaire
3. Réseaux multi-couches
4. Différentiation automatique

- On ne gagne rien en rajoutant des couches de transmissions affines, puisque la composée de fonctions affines est une fonction affine



$$x_{L+1} = W_L x_L + b_L = \dots = (W_L \circ \dots \circ W_0) x_0 + \sum_{k=0}^L (W_L \circ \dots \circ W_{k+1}) b_k$$

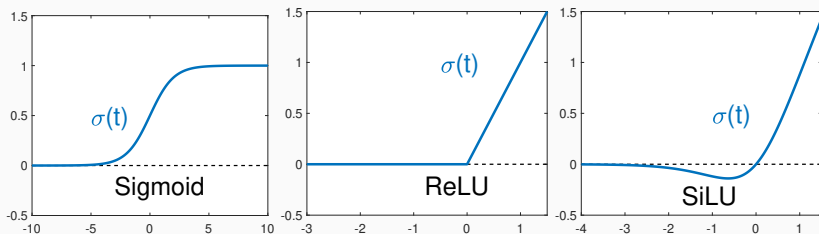
- Pour gagner en **expressivité**, on introduit des **non-linéarités** dans **certaines** couches

$$x_{k+1} := \sigma(W_k x_k + b_k)$$

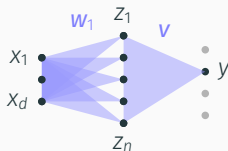
où  $\sigma$  est une fonction non-linéaire, que l'on applique à chaque neurone. Les réseaux résultants  $(\theta_0, \theta_1, \dots, \theta_L)$  (i.e. **entièrement connectés + non-linéarités**) s'appellent des **perceptrons**.



- Pour les fonctions non-linéaires  $\sigma$ , on parle de **fonctions d'activation**.
- Pour une meilleure expressivité, on les choisit non-polynomiale



# THÉORÈME D'APPROXIMATION UNIVERSELLE



$$y = \sum_{k=1}^n v_k \sigma(\mathbf{w}_k^T \mathbf{x} + b_k) + c_k$$

- Un perceptron avec seulement une couche cachée (mais de **largeur arbitraire**) permet d'approcher n'importe quelle fonction continue (sur un compact)

**Theorem 1 (Cybenko).** Soit  $f : \mathcal{C} \mapsto \mathbb{R}$  une fonction continue. Alors pour tout  $\varepsilon > 0$ , il existe  $n \in \mathbb{N}$  et  $\theta = (\mathbf{W}, \mathbf{b}, \mathbf{v}) \in \mathbb{R}^{n \times d} \times \mathbb{R}^n \times \mathbb{R}^d$  tels que

$$\forall \mathbf{x} \in \mathcal{C}, \quad |f_{\theta}(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$$

1. Introduction
2. Modèle linéaire
3. Réseaux multi-couches
4. Différentiation automatique





- L'apprentissage statistique ne se réduit pas à de la classification binaire supervisée
- **Classification supervisée, semi-supervisée, non supervisée**: méthode des K plus proches voisins (K-means), forêt d'arbres décisionnels (random forest), SVM multi-classes, réseaux de neurones
- Régression